

Section 2.1: Networking Models, Protocols & Port Configurations

Enhanced Edition — Additional examples, comparisons, and interview preparation

Executive Overview

Network protocols define how systems communicate. Understanding OSI layers, transport protocols, and addressing schemes is essential for designing scalable, reliable systems. Interviewers probe for depth in TCP mechanics, NAT traversal, and protocol selection.

How to use this section: Every network design decision is a trade-off between latency, reliability, ordering, and overhead. The frameworks here — layer model, TCP vs. UDP, NAT types — give you a vocabulary to reason through those trade-offs under interview pressure.

Interview mindset: When asked about a networking problem, always anchor your answer to the OSI layer it lives at. “This is a Layer 4 problem — it’s about connection management, not application semantics” signals clarity of thought that interviewers reward.

OSI Model vs. TCP/IP Stack

What It Is

The OSI model provides a theoretical framework for reasoning about network behavior; TCP/IP describes the practical internet stack. Understanding both is essential — OSI is the *language* you use in interviews; TCP/IP is what actually runs in production.

Layer Mapping

OSI 7-Layer Model with real-world artefacts at each layer:

Layer 7: Application	HTTP, SMTP, FTP, DNS, gRPC
Layer 6: Presentation	TLS/SSL, JSON encoding, compression (gzip)
Layer 5: Session	TCP connection setup/teardown, WebSocket framing
Layer 4: Transport	TCP (reliable), UDP (unreliable), port numbers
Layer 3: Network	IP routing, ICMP (ping), BGP, OSPF

Layer 2: Data Link	Ethernet frames, MAC addressing, ARP, VLANs
Layer 1: Physical	Fiber, copper, 5G radio, voltage levels

TCP/IP 4-Layer Model (what actually runs):

Layer 4: Application	HTTP/2, DNS, gRPC (merges OSI 5-7)
Layer 3: Transport	TCP segments, UDP datagrams
Layer 2: Internet	IP packets (IPv4/IPv6)
Layer 1: Link	Ethernet frames (merges OSI 1-2)

Why the Layer Model Matters in Interviews

Each layer boundary is a decision point. When you say “I’ll put the load balancer here,” you’re implicitly choosing a layer:

Component	OSI Layer	What It Can See / Do
Network switch	L2	MAC addresses only; no IP visibility
Router / firewall	L3	IP source/destination; port numbers hidden
L4 load balancer (NLB)	L4	TCP/UDP ports; cannot inspect HTTP headers
L7 load balancer (ALB)	L7	Full HTTP: URL path, cookies, headers
API Gateway	L7	JWT tokens, API keys, JSON body
Service mesh (Envoy)	L4–L7	mTLS identity, circuit breaking, retries

The practical consequence: An L4 load balancer is faster (no HTTP parsing) but cannot route `/api/users` to UserService and `/api/orders` to OrderService — that requires L7 inspection.

Debugging Network Issues — Layer-by-Layer Decision Tree

```

Service unreachable?
|
├─ L1 Physical: is NIC up? → ip link show
├─ L2 Data Link: ARP resolved? → arp -n; can you reach gateway?
├─ L3 Network: can you ping? → ping 10.0.0.1; traceroute
├─ L4 Transport: port open? → telnet host 443; nc -zv host 443
├─ L5 Session: TLS handshake? → openssl s_client -connect host:443
├─ L6 Presentation: cert valid? → check CN/SAN, expiry, chain
└─ L7 Application: HTTP 200? → curl -v https://host/health

```

Transport Layer Protocol Mechanics

TCP Deep Dive

TCP guarantees **ordered, reliable, error-checked** delivery. Every feature that makes it reliable adds latency — understanding each trade-off is essential for protocol selection interviews.

Three-Way Handshake — annotated:

```
Client                                     Server
|— SYN (seq=100) —————→| "I want to connect, starting seq at 100"
|← SYN-ACK (seq=200, ack=101) ————| "OK, I'm starting at 200, I got your
100"
|— ACK (seq=101, ack=201) —————→| "I got your 200. Connection open."
|
| Total: 1 RTT before first byte
| Cross-continent (SF→London):
| ~80ms RTT → 80ms before HTTP req |
```

Then add TLS on top (TLS 1.2, pre-2018):

```
TCP handshake:    1 RTT  (80ms)
TLS 1.2 handshake: 2 RTT  (160ms)
First HTTP byte:  1 RTT  (80ms)
-----
Total first byte: 4 RTT = 320ms cross-continent
```

This is why TLS 1.3 and QUIC exist — they collapse this to 1–2 RTT total.

Flow Control (Sliding Window):

```
Receiver advertises buffer size (rwnd) in every ACK:

Sender:  [sent+unacked]———[can send]———[blocked]
         ←—— cwnd ———→ ←—— rwnd-cwnd —→

If rwnd = 0: sender pauses (receiver buffer full)
If packet lost: sender halves cwnd (congestion signal)
```

TCP Congestion Control states:

```
Slow Start (cwnd doubles per RTT):
  1 MSS → 2 → 4 → 8 → 16 → ssthresh

Congestion Avoidance (cwnd += 1 MSS per RTT after ssthresh):
  16 → 17 → 18 → 19 ...
```

```
3× Duplicate ACKs → Fast Retransmit + Fast Recovery:  
ssthresh = cwnd/2; cwnd = ssthresh + 3  
(faster recovery than timeout, which resets cwnd to 1)
```

```
Timeout (worst case):  
cwnd = 1 MSS; back to Slow Start
```

Real-world implication: A file transfer from SF to London that experiences one timeout drops throughput from ~10 Mbps back to near zero for several RTTs — a classic long-fat-network problem. Solution: BBR congestion control (used by Google), which models bandwidth directly rather than reacting to loss.

TCP Congestion Control Algorithm Comparison:

Algorithm	Strategy	Recovery Speed	When to Use	Used By
Reno (1990s)	Loss-based, halves cwnd on loss	Slow	Legacy systems only	Deprecated
Cubic (Linux default)	Loss-based, cubic growth	Medium	General internet, most servers	90%+ of internet
BBR (Google, 2016)	Model-based (bandwidth + RTT)	Fast	High BDP networks, lossy links	YouTube, Google services
Vegas	Delay-based (detects congestion early)	Very conservative	Low-latency networks	Rarely used (too conservative)

Deep dive — Why BBR matters:

```
Cubic behavior on 1% packet loss (satellite, cellular):  
Sees loss → halves cwnd → throughput drops 50%  
But loss wasn't from congestion, just noisy channel  
Result: Underutilizes available bandwidth
```

```
BBR behavior on same link:  
Models max bandwidth and min RTT independently  
Doesn't react to non-congestion loss  
Result: Maintains near-optimal throughput
```

```
Performance delta: BBR achieves 2-10× higher throughput on lossy links
```

When to use each: - **Cubic**: Default for most scenarios, battle-tested, fairness proven - **BBR**: High bandwidth-delay product (BDP) networks (satellite, intercontinental), mobile/WiFi (lossy but not congested) - **Switch from Cubic to BBR when**: throughput $\ll$ bandwidth $\times$ RTT, high packet loss but not from congestion

Interview insight: Mentioning BBR shows you understand modern congestion control goes beyond loss-based algorithms.

Head-of-Line (HoL) Blocking — the core TCP weakness:

HTTP/1.1 pipeline: Request A $\rightarrow$ Request B $\rightarrow$ Request C

If packet for Request A is lost:

B and C wait in TCP receive buffer even though their data arrived
All streams stalled until A is retransmitted and reordered

HTTP/2 fix: multiple streams over one TCP connection

$\rightarrow$ still has TCP-level HoL blocking (one lost packet stalls all streams)

QUIC/HTTP/3 fix: multiple streams over UDP

$\rightarrow$ lost packet only stalls its own stream, not others

QUIC Deep Dive — Why HTTP/3 Matters:

Feature	TCP + TLS 1.2	TCP + TLS 1.3	QUIC (HTTP/3)
Connection setup	3 RTT (TCP + TLS)	2 RTT	0-1 RTT (0-RTT on resumption)
HoL blocking	Yes (TCP-level)	Yes (TCP-level)	No (per-stream)
Connection migration	No (breaks on IP change)	No	Yes (survives WiFi $\rightarrow$ Cellular)
Multiplexing	HTTP/2 streams	HTTP/2 streams	Native streams
Packet loss impact	All streams stall	All streams stall	Only affected stream stalls

QUIC benefits explained:

1. 0-RTT Connection Resumption:

First connection:

Client $\rightarrow$ Server: ClientHello + cached params

Server $\rightarrow$ Client: Response with data immediately

Total: 1 RTT to first byte (vs 3 RTT for TCP+TLS 1.2)

Subsequent connections (within session ticket validity):
Client → Server: 0-RTT data + session ticket
Server → Client: Response immediately
Total: 0 RTT to first byte

Use case: Mobile apps that reconnect frequently save 100-300ms per reconnect

2. Connection Migration (critical for mobile):

User on WiFi (IP: 192.168.1.100) → downloads file
Walks outside, switches to cellular (IP: 10.20.30.40)

TCP behavior: Connection breaks, must re-establish (3 RTT penalty)
QUIC behavior: Connection continues seamlessly via connection ID

Real impact: YouTube reports 30% fewer rebuffers on QUIC vs TCP

3. Per-Stream Independence:

Loading webpage with 10 resources over HTTP/2:
- CSS packet lost → entire page render stalls

Same page over HTTP/3 (QUIC):
- CSS packet lost → only CSS stream stalls
- Images, JS continue loading independently

When to use QUIC/HTTP/3: -  High packet loss environments (mobile, WiFi) -  Frequent reconnections (mobile apps) -  Multiplexed streams where independence matters (web pages) - 
Controlled datacenter environments (TCP is fine) -  When middleboxes block UDP (some corporate firewalls)

Adoption status (2026): - Google services: 100% QUIC - Cloudflare, Fastly: Full support - AWS CloudFront: Supported - Browser support: Chrome, Firefox, Safari (all major browsers)

TCP vs. UDP — Protocol Selection Guide

Property	TCP	UDP
Connection	3-way handshake required	Connectionless
Delivery	Guaranteed, ordered	Best-effort, unordered
Error recovery	Automatic retransmit	Application must handle
Flow control	Yes (rwnd)	No

Property	TCP	UDP
Congestion control	Yes (cwnd)	No
Overhead per packet	20-byte header	8-byte header
Use case	Web, DB, file transfer	DNS, video, gaming, VoIP

UDP in practice — it's not “just fire and forget”:

Real-world protocols built on UDP add selective reliability:

DNS (pure UDP, no reliability):

Query → Response in 1 packet; if lost, client retries after 5s timeout

RTP (Real-time Transport Protocol, for video/audio):

Sequence numbers for ordering detection

Timestamp for jitter calculation

No retransmit — late packet is worse than no packet for real-time media

QUIC (built on UDP):

Adds its own reliability, ordering, congestion control

But per-stream — lost packet doesn't stall other streams

Used by YouTube, Google Search, Cloudflare

UDP Reliability Patterns — Building on Top of UDP:

Pattern	Reliability Level	Ordering	When to Use	Example
Pure UDP (no reliability)	None	No	Single packet exchanges, stateless	DNS queries
Timeout + Retry	Basic	No	Simple request-response	DNS, NTP, SNMP
Selective ACK	High	Yes	Custom reliability needed	QUIC, SCTP over UDP
FEC (Forward Error Correction)	Medium	No	Real-time where retransmit too slow	Live video streaming
Hybrid (FEC + Selective Retransmit)	High	Yes	Real-time with quality needs	WebRTC, Zoom

Deep dive on each pattern:

1. **Timeout + Retry** (simplest):

```
def udp_request_with_retry(host, port, data, timeout=5, retries=3):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.settimeout(timeout)

    for attempt in range(retries):
        sock.sendto(data, (host, port))
        try:
            response, _ = sock.recvfrom(1024)
            return response
        except socket.timeout:
            if attempt == retries - 1:
                raise
    return None
```

Use case: DNS, simple monitoring probes

2. **Selective ACK** (QUIC-style):

```
Sender sends packets: 1, 2, 3, 4, 5
Packet 3 is lost

Receiver ACKs: "Got 1, 2, 4, 5; missing 3"
Sender retransmits only packet 3

vs. TCP: "Got 1, 2; waiting for 3 before acknowledging 4, 5"
```

Benefit: Only retransmit what's lost, not everything after **Trade-off:** More complex state management

3. **FEC (Forward Error Correction)**:

```
Send 10 data packets + 3 parity packets
Receiver can reconstruct original 10 from any 10 of the 13

Result: Can tolerate 3 packet losses without retransmit
Cost: 30% bandwidth overhead
```

Use case: Live streaming where 50ms retransmit = too late **Example:** WebRTC uses FEC for critical keyframes

4. **Hybrid approach** (modern real-time apps):

```
For audio/video frames:
- Use FEC to tolerate burst losses (no RTT penalty)
```


- If FEC can't recover, request retransmit
- Decoder conceals gaps if retransmit arrives too late

Used by: Zoom, Teams, Discord voice chat

Decision framework:

Packet loss tolerance?

- └ 0% acceptable → TCP
- └ <1% acceptable, ordering critical → QUIC
- └ <5% acceptable, real-time → UDP + FEC
- └ >5% acceptable, best-effort → Pure UDP

Retransmit latency acceptable?

- └ Yes (>50ms OK) → UDP + Selective ACK
- └ No (<20ms required) → UDP + FEC only

Interview insight: Saying “I’d use UDP with selective ACK and FEC for the keyframes” shows you understand UDP isn’t just unreliable—it’s a foundation for custom reliability models.

Decision rule for interviews: - Use TCP when: data must arrive, order matters, application doesn’t want to manage reliability - Use UDP when: latency > reliability (real-time media), app implements its own reliability (QUIC), DNS-style stateless single-packet exchanges

Common Port Numbers — Know These Cold

Port	Protocol	Used By
22	SSH	Remote shell, SCP, SFTP
53	DNS	Name resolution (TCP+UDP)
80	HTTP	Web traffic
443	HTTPS	TLS-encrypted web
3306	MySQL	Database
5432	PostgreSQL	Database
6379	Redis	Cache / pub-sub
9092	Kafka	Message broker
2181	ZooKeeper	Coordination
8080 / 8443	HTTP(S) alt	App servers, proxies

Interview tip: Saying “I’d open port 443 at the security group and terminate TLS at the load balancer, leaving port 80 for health checks on the internal network” shows systems fluency that raw protocol knowledge alone doesn’t.

IP Addressing Architectures

IPv4 vs. IPv6

IPv4 Exhaustion — why it matters operationally:

```
Total IPv4 space:    ~4.3 billion addresses
Globally assigned:   ~3.7 billion
Private ranges:
  10.0.0.0/8         → 16.7M addresses (large enterprises, AWS VPCs)
  172.16.0.0/12      → 1M addresses (Docker default: 172.17.0.0/16)
  192.168.0.0/16     → 65K addresses (home networks)

Result: NAT is mandatory for any multi-device network with a single ISP IP
```

IPv6 Design Advantages:

```
128-bit addresses: 3.4 × 1038 total
= 340 undecillion = enough for 1028 addresses per human alive today

Address format (abbreviated):
Full:      2001:0db8:85a3:0000:0000:8a2e:0370:7334
Compressed: 2001:db8:85a3::8a2e:370:7334  (:: = consecutive zero groups)

Header improvements over IPv4:
Fixed 40-byte header (IPv4 is variable 20-60 bytes)
No header checksum (link layer handles it → faster routing)
Extension headers (optional, chained): fragmentation, routing, auth
Flow labels: QoS without deep packet inspection

Native features IPv4 lacked:
Stateless Address Autoconfiguration (SLAAC) — no DHCP needed
Mandatory IPSec support (optional in IPv4)
Multicast replaces broadcast
```

CIDR Notation Refresher — interviewers test this in VPC design questions:

```
10.0.0.0/16 → 65,536 addresses (10.0.0.0 – 10.0.255.255)
10.0.0.0/24 → 256 addresses (typical subnet)
10.0.0.0/28 → 16 addresses (small subnet, e.g., load balancers only)

AWS VPC design pattern:
```

VPC:	10.0.0.0/16	(65K total)
Public AZ-A:	10.0.0.0/24	(256 – NAT gateway, ALB)
Public AZ-B:	10.0.1.0/24	
Private AZ-A:	10.0.10.0/24	(256 – app servers)
Private AZ-B:	10.0.11.0/24	
DB AZ-A:	10.0.20.0/24	(256 – RDS, ElastiCache)
DB AZ-B:	10.0.21.0/24	

IPv6 Transition Mechanisms Compared

Mechanism	How It Works	Best For	Limitation
Dual-Stack	Each host has both IPv4 and IPv6	Gradual migration	Doubles protocol complexity
6to4 tunneling	IPv6 packets wrapped in IPv4	IPv6 islands over IPv4 WAN	Deprecated; MTU issues
NAT64 + DNS64	IPv6-only clients reach IPv4 servers via gateway	Mobile-only IPv6 networks	Gateway is a single point of failure
464XLAT	Client-side CLAT translates IPv4 apps → IPv6	iOS/Android on IPv6-only networks	Requires PLAT (NAT64) on carrier side

Why this matters for mobile: Apple requires all iOS apps to support IPv6-only networks (since 2016). If your service has an AAAA record and your CDN supports IPv6, iOS clients on carrier IPv6 networks get direct IPv6 connectivity without going through a NAT64 gateway.

Network Address Translation (NAT)

What It Is

NAT rewrites IP addresses (and ports) as packets cross the gateway, allowing many private-IP devices to share one public IP. It is simultaneously a scaling mechanism, a security boundary, and a source of P2P connectivity pain.

NAT Types — the P2P Compatibility Spectrum

Most permissive					Most restrictive
Full-Cone	→	Restricted-Cone	→	Port-Restricted	→
					Symmetric

Easy P2P

P2P impossible

Full-Cone NAT (rare, home routers with UPnP):

Internal: 192.168.1.5:5000 → External: 203.0.113.1:40001

Rule: ANY external host can send to 203.0.113.1:40001
→ P2P trivial, but security-weak

Restricted-Cone NAT (common home routers):

Rule: External host X can send to the mapped port
ONLY if 192.168.1.5 previously sent TO host X

→ P2P works if one peer initiates outbound first (hole-punching)

Port-Restricted NAT (stricter home routers):

Rule: External host X:port Y can send to mapped port
ONLY if 192.168.1.5 previously sent TO host X:port Y specifically

→ P2P hole-punching still works but must match exact port

Symmetric NAT (enterprise firewalls, AWS NAT Gateway):

Internal: 192.168.1.5:5000 → Server A: external port 40001

Internal: 192.168.1.5:5000 → Server B: external port 40002 ← DIFFERENT port!

→ STUN discovers port 40001 but WebRTC attempts to contact via 40001
which doesn't accept packets from Server B
→ P2P impossible; must fall back to TURN relay

NAT Traversal — STUN, TURN, ICE

STUN (Session Traversal Utilities for NAT):

Client → STUN Server: "What is my public IP:port?"
STUN Server → Client: "You appear as 203.0.113.1:40001"

Works for: Full-cone, restricted-cone, port-restricted
Fails for: Symmetric NAT (each destination gets a different port)
Used by: WebRTC, VoIP, gaming

TURN (Traversal Using Relays around NAT):

Client A → TURN Server → Client B
(relay)

Always works: TURN is a server relay, bypasses NAT entirely

Cost: All media traffic routes through TURN server

→ higher latency, bandwidth cost

Used by: WebRTC fallback when STUN fails (~15-20% of calls)

ICE (Interactive Connectivity Establishment) — the full strategy:

1. Gather candidates:
 - Host candidate: 192.168.1.5:5000 (direct LAN)
 - Server-reflexive: 203.0.113.1:40001 (via STUN)
 - Relay: 198.51.100.5:50001 (via TURN)
2. Exchange candidates with peer (via signaling server)
3. Connectivity checks (STUN binding requests to each candidate pair)
4. Select best working path:
Preference: Host > Server-reflexive > Relay
5. Result: direct P2P if possible, TURN relay if not

ICE is why WebRTC calls succeed even through NAT — it systematically tries every connectivity option and falls back gracefully.

Interview Questions

★ Protocol Selection

When would you use UDP instead of TCP for a backend service? Give a concrete example.

Solution Framework

Key principle: Use UDP when the cost of retransmission or ordering exceeds the cost of handling unreliable delivery at the application layer.

Concrete examples:

1. **DNS** — pure UDP, single packet, <512 bytes:

Resolver → Query → DNS Server
DNS Server → Response (UDP, best-effort)

If no response in 5s: resolver retries

Why not TCP? A DNS query fits in one UDP packet.
TCP handshake would cost more than the query itself.

2. **Game state synchronization** (e.g., player positions in a multiplayer game):

Server broadcasts positions 60×/second
A position update that's 16ms late is useless – just discard it
TCP would buffer and deliver stale position after a loss event
→ Player sees a "rubber-band" effect (teleportation)

UDP with sequence numbers: if seq N+1 arrives, drop seq N
→ Smooth real-time experience even with occasional packet loss

3. **Metrics aggregation** (StatsD protocol):

Application → UDP → StatsD Daemon → Aggregator

"Counter increments dropped during a network blip" is acceptable
The monitoring system survives; the business doesn't depend on exact counts
TCP overhead on every metric emission would slow the application

When NOT to use UDP: - Financial transactions (you need exactly-once delivery guarantee) - File transfers (data integrity required) - Database connections (ordered queries, reliable responses mandatory)

Interview signal: Frame your answer as “UDP makes sense when either (a) a dropped packet is cheaper than a delayed packet, or (b) the application can implement its own reliability more efficiently than TCP.”

☆☆ **TCP Latency Optimization**

Explain why TCP's three-way handshake adds ~150ms latency to a cross-continent request. How do modern protocols address this, and what are the trade-offs of each approach?

Solution Framework

Latency breakdown (SF → London, ~80ms RTT):

Pure TCP:	
SYN → SYN-ACK → ACK:	1 RTT = 80ms
First HTTP request:	1 RTT = 80ms
Total first byte:	2 RTT = 160ms

TCP + TLS 1.2:

```

TCP handshake:          1 RTT = 80ms
TLS handshake (ClientHello...): 2 RTT = 160ms
First application byte: 1 RTT = 80ms
Total:                  4 RTT = 320ms

```

TCP + TLS 1.3 (new session):

```

TCP handshake:          1 RTT = 80ms
TLS 1.3 handshake:      1 RTT = 80ms
First application byte: 1 RTT = 80ms
Total:                  3 RTT = 240ms

```

QUIC/HTTP/3 (new session):

```

QUIC handshake (combines TCP+TLS): 1 RTT = 80ms
First application byte: 0 RTT
Total:                    1 RTT = 80ms

```

Protocol solutions with trade-offs:

Approach	Latency Saving	Security Trade-off	Deployment Complexity
TLS 1.3 (1-RTT)	Saves 1 RTT vs TLS 1.2	None — safer than 1.2	Low (config change)
TLS 1.3 0-RTT resumption	Saves 1 RTT on reconnect	Replay attacks on non-idempotent requests	Medium (ticket management)
TCP Fast Open	Saves 1 RTT for SYN	None	Low (OS flag) — but limited support
QUIC/HTTP/3	Saves 1–3 RTT total	None — TLS 1.3 mandatory	High (new protocol stack)

0-RTT replay attack explained:

0-RTT: Client sends app data with the SYN (using session ticket)

Attack: Attacker captures 0-RTT packet and replays it
 → POST /payment/charge replayed = double charge

Mitigation: Only use 0-RTT for idempotent requests (GET, HEAD)
 Reject 0-RTT for state-mutating operations

QUIC connection migration (killer feature):

TCP: connection identified by (src_ip, src_port, dst_ip, dst_port)
 → IP change (WiFi → 5G) = new connection = re-handshake

QUIC: connection identified by 64-bit connection ID

- IP change → same connection continues
- No interruption for mobile users switching networks

Key insight: Network latency is ultimately physics-bound — the speed of light across 8,000km cannot be reduced. Protocol optimizations reduce *handshake count*, not the speed of a single packet. CDN edge nodes solve the remaining gap by terminating connections close to the user.

☆☆☆ Global Video Call Architecture

Design the networking layer for a video calling service (like Zoom) that supports 1M simultaneous calls. Address NAT traversal, protocol selection, and global routing.

Solution Framework

Constraints analysis:

1M simultaneous calls × 2 participants = 2M endpoints
Each call: ~1 Mbps video + 64 Kbps audio ≈ 1.1 Mbps bidirectional
Total bandwidth if all P2P: 2M × 1.1 Mbps ≈ 2 Tbps peak
~20% of calls will fail ICE P2P and need TURN relay
TURN load: 200K calls × 2.2 Mbps = 440 Gbps relay bandwidth

Protocol selection:

Media: RTP over UDP (real-time, no retransmit — latency > reliability)
With SRTP (Secure RTP) for encryption
RTCP for statistics (packet loss, jitter, RTT)

Signaling: WebSocket over TLS (reliable, real-time updates)
Call setup, ICE candidate exchange, codec negotiation

P2P connectivity: ICE (STUN + TURN)
STUN: discover public IP:port (self-hosted in each region)
TURN: relay fallback (~20% of calls)

NAT traversal strategy:

1. Both clients query regional STUN server → get reflexive candidates
2. Exchange candidates via signaling server
3. ICE connectivity checks (simultaneous hole-punching):
 - Client A → Client B using B's reflexive candidate
 - Client B → Client A using A's reflexive candidate
 - Both packets arrive "simultaneously", opening the hole

4. If symmetric NAT: ICE falls back to TURN relay
5. Select best candidate pair: Host > Reflexive > Relay

Global routing architecture:

Internet
↓
GeoDNS → Regional TURN/Relay Cluster (5 regions)
↓
Selective Forwarding Unit (SFU) per region
↓
Call participants

SFU pattern (vs. P2P mesh):
P2P mesh (n participants): each sends n-1 streams = $O(n^2)$ total
SFU: each sends 1 stream to SFU; SFU fans out to others = $O(n)$ total

For group calls (>2 participants): SFU is essential
For 1:1 calls: direct P2P preferred (lower latency, no server cost)

Regional TURN sizing:

200K concurrent TURN calls × 2.2 Mbps = 440 Gbps
Per region (5 regions): ~88 Gbps
TURN server bandwidth: 10 Gbps per server → 9 servers per region
With 2× headroom: 18 TURN servers per region × 5 = 90 globally

Cost implication: TURN is expensive → optimize STUN success rate
Strategies: UDP port 443 (mimics HTTPS, bypasses many firewalls),
TCP fallback on port 443

Jitter buffer — the real-time quality lever:

Network jitter: packets arrive with variable delay
Without buffer: audio choppy (speaker sounds like "he-llo-- ho-w-a-re")
With buffer: hold packets, play at fixed rate, smooth output

Buffer size trade-off:
Small buffer (20ms): low latency, choppy under bad network
Large buffer (150ms): smooth, but conversation feels laggy

Adaptive buffer: expand when jitter increases, shrink when stable
→ Zoom/Meet use adaptive 20-150ms range

Key insights: 1. ICE is the backbone of NAT traversal — understand host/reflexive/relay candidates and the preference order. 2. SFU architecture is necessary for group calls; P2P is preferred for 1:1. 3. TURN relay is the cost driver — every dollar spent improving STUN success rate saves 10x in

relay costs. 4. UDP is mandatory for low-latency media — TCP's retransmit backpressure ruins real-time audio.

Quick Reference: Networking Interview Cheat Sheet

When asked about...	Key points to hit
TCP vs. UDP	Reliability vs. latency; ordered vs. best-effort; when dropped packet < stale packet
Three-way handshake	1 RTT cost; SYN queue; SYN flood attacks; QUIC eliminates it
Head-of-line blocking	TCP: stream-level; HTTP/2: TCP-level still; QUIC: per-stream only
NAT traversal	Symmetric NAT kills P2P; ICE = STUN + TURN; hole-punching
IPv6 adoption	Dual-stack; NAT64 for mobile; SLAAC; 128-bit no-NAT-needed
Protocol for real-time	UDP + RTP/SRTP; adaptive jitter buffer; no retransmit
Latency from SF to London	~80ms RTT; 4 RTT for TCP+TLS 1.2; 1 RTT for QUIC